



THE UNIVERSITY *of* EDINBURGH
School of Biological Sciences

‘PyPollen’ – Development of a Python
module towards the automation of pollen
species identification to combat pollinator
decline

Ben Oscroft

Sup: Prof. Maurice Gallagher & Dr. Chris Wood

Biological Sciences (Biotechnology)

BSc. Hons

Exam Number: B122267

Table of Contents

TABLE OF CONTENTS	1
TABLE OF FIGURES	2
GLOSSARY & ABBREVIATIONS	ERROR! BOOKMARK NOT DEFINED.
1. ABSTRACT	ERROR! BOOKMARK NOT DEFINED.
2. INTRODUCTION.....	ERROR! BOOKMARK NOT DEFINED.
2.1 POLLINATOR DECLINE	ERROR! BOOKMARK NOT DEFINED.
2.2 POLLEN MORPHOLOGY	ERROR! BOOKMARK NOT DEFINED.
2.3 CURRENT RESEARCH & AIMS.....	ERROR! BOOKMARK NOT DEFINED.
3. METHODOLOGY	ERROR! BOOKMARK NOT DEFINED.
3.1 INITIAL DESIGN.....	ERROR! BOOKMARK NOT DEFINED.
3.2 CHOICE OF TECHNIQUES/RESOURCES	ERROR! BOOKMARK NOT DEFINED.
3.2.1 <i>Techniques</i>	<i>Error! Bookmark not defined.</i>
3.2.2 <i>Selection of modules and resources</i>	<i>Error! Bookmark not defined.</i>
3.3 PROCESS & IMPLEMENTATION	ERROR! BOOKMARK NOT DEFINED.
3.3.1 <i>Data collection</i>	<i>Error! Bookmark not defined.</i>
3.2.2 <i>Image segmentation</i>	<i>Error! Bookmark not defined.</i>
3.2.3 <i>Analysis techniques</i>	<i>Error! Bookmark not defined.</i>
4. RESULTS.....	ERROR! BOOKMARK NOT DEFINED.
4.1 SEGMENTATION	ERROR! BOOKMARK NOT DEFINED.
4.1.1 <i>Particle detection</i>	<i>Error! Bookmark not defined.</i>
4.1.2 <i>Feature extraction</i>	<i>Error! Bookmark not defined.</i>
4.2 MACHINE LEARNING & DATA POLISHING	ERROR! BOOKMARK NOT DEFINED.
4.2.1 <i>PCA</i>	<i>Error! Bookmark not defined.</i>
4.2.3 <i>Data enhancement</i>	<i>Error! Bookmark not defined.</i>
4.2.2 <i>K-means clustering</i>	<i>Error! Bookmark not defined.</i>
4.2.3 <i>Support Vector Machine (SVM) supervised learning</i>	<i>Error! Bookmark not defined.</i>
5. DISCUSSION	ERROR! BOOKMARK NOT DEFINED.
5.1 PERFORMANCE	ERROR! BOOKMARK NOT DEFINED.
5.2 CRITIQUE.....	ERROR! BOOKMARK NOT DEFINED.
5.3 FUTURE STEPS	ERROR! BOOKMARK NOT DEFINED.
6. CONCLUSION	ERROR! BOOKMARK NOT DEFINED.
ACKNOWLEDGEMENTS	ERROR! BOOKMARK NOT DEFINED.
BIBLIOGRAPHY	ERROR! BOOKMARK NOT DEFINED.
APPENDIX I – <i>PALDAT</i> WEB SCRAPING SCRIPT	ERROR! BOOKMARK NOT DEFINED.
APPENDIX II – ‘ <i>PYPOLLEN</i> ’ CODE	ERROR! BOOKMARK NOT DEFINED.

Table of Figures & Tables

Table 1 – Anthropogenic factors which contribute to pollinator decline
Table 2 – Classes of morphological features of pollen
Figure 1 – Illustrated morphological features of pollen
Figure 2 – Initial software design
Table 3 – Python libraries used in development
Figure 3 – Examples of dataset images
Figure 4 – Segmentation process flow chart
Figure 5 – Grain detection scatter plots
Figure 6 – Grain size scatter plot
Figure 7 – Magnification factor histogram
Table 4 – Magnification factor statistics
Figure 8 – Magnification-adjusted grain size scatter plot
Figure 9 – 2-component PCA plots of polarity
Figure 10 – K-Means clustering with labelled errors
Table 5 – Full results of machine learning models

Glossary & Abbreviations

Palynological – (relating to) the study of pollen

Morphology – The form, shape, or structure of something

Forage – The food supply for bees within their locality

Segmentation – Partitioning of an image into regions of interest

Mask – Filter applied to prevent functions from being applied to pre-defined areas of an image

Python module/library/package – Code containing defined functions which can be imported and used in other scripts

Web scraping – Technique to extract information from HTML files (such as websites)

For definitions specific to pollen morphology and prefixes please see Punt, *et al*, [29] or Halbritter, *et al*. [11]

Abbreviations

LM – Light Microscopy

ML – Machine Learning

GPP – Global Pollen Project

SAPS – Schools And Plants for Schools

GLCM – Grey-Level Co-occurrence Matrix

CNN – Convolutional Neural Network

ARI – Adjusted Rand Index

SEM – Standard Error of the Mean

SD – Standard Deviation

1. Abstract

The decline of insect pollinator species populations is an impending threat to food security, primarily due to anthropogenic factors such as global warming and the intensification of agriculture. 'PyPollen' is a Python module which creates the framework for software that is designed to automatically identify species of pollen from light microscopy (LM) images, a widely recognised requirement in combating pollinator decline, using a new approach which aims to identify morphological features rather than species directly. Image segmentation code was developed with implemented machine learning (ML) models to improve its function by predicting errors. This ML model, as well as models for predicting pollen features, was trained using a data set of 4,099 images and annotated with a database of morphological descriptions, gathered using 'web scraping'. While the models for classifying features were unreliable, the error prediction model showed a high accuracy of 81%. More high-quality data and more sophisticated ML architectures are required to further develop this software.

2. Introduction

2.1 Pollinator decline

Pollination, the mechanism by which plants sexually reproduce, is paramount to the survival of plant species from novel selection pressures, and as such is intrinsic to food security. While many crops, including major cereals, are pollinated by wind (anemophily), 70-86% of crop species benefit from insect pollination (entomophily), with 35%, which are often major sources of micro- and macronutrients, entirely dependent on it [17]. Such crops include soybeans, coffee, potatoes, peanuts, sugarcane, citrus, pulses, and cotton. Insects such as moths, beetles, and butterflies are considered pollinators [44], but bees are widely regarded as the 'elite class'; while bees comprise only 2% of insect pollinator species, they account for 80% of crop visits [16]. This propensity for pollination can be attributed to their diet consisting entirely of pollen and nectar, hence have evolved to collect as much pollen as possible. Bees are 'generalist' pollinators, meaning they visit many species, but exhibit a temporary loyalty to one species at a time, as these shall all be in bloom simultaneously, thereby increasing the chance of compatible pollination [44]. Colonies of hundreds of thousands of bees intelligently communicate the location of pollen-rich plants to each other, with estimates gauging that each bee visits up to 14,000 flowers per day [3].

'Pollinator decline', the gradual reduction of pollinator populations, has been recognised as a major issue since the 1980s, particularly in North America and Europe, with 41% of pollinator species' population decreasing by 1% per annum. While the cause remains elusive, it is thought anthropogenic factors are largely to blame [32], as displayed in table 1.

Drivers of pollinator decline	Anthropogenic influence	Effect on pollinators	Weight of factor on pollinator decline
Intensive agriculture/ Urbanisation	Destruction of habitat and fragmentation of land into increasingly low-diversity landscapes	- Specialist pollinators are replaced by invasive generalists as their niche is removed - Beehives require space to thrive, and habitat invasion increases competition between pollinators	34.6%
Monoculture farming	Replacement of natural diverse flora with genetically identical crops	- Less diverse pollinator diet reduces immunocompetency and nutrition - Monoculture creates conditions for pests to flourish - Foraging window for pollinators is reduced – In diverse ecosystems, blooming periods overlap	(Included in intensive agriculture)
Pesticide/ Herbicide/ Fertiliser pollution	Seepage of synthetic chemicals into natural ecosystems	- Many chemicals are directly toxic to insects - Some also disrupt insect navigation - Herbicides reduce biodiversity and disturb natural ecosystems	22.7%
Ecological influence	Factors affecting pollinator's ability to survive natural selection pressures	- Introduction of invasive species increases interspecies competition, decimating endemic species populations - These also carry novel pathogens pollinators which may already be immunocompromised - Attempts at afforestation have reduced diversity and damaged insect populations - Hives are sometimes relocated to pollinate crops in areas where the population has been reduced, the stress of which can destroy whole colonies	12.6%
Deforestation	Rampant deforestation in tropical countries for lumber and urban expansion	Direct destruction of habitats and species kills endemic and specialist species of pollinators	8.8%
Climate change	Rising global temperatures and erratic weather events due to anthropogenic greenhouse gas emissions	- Rising temperatures have directly caused some thermally intolerant species to suffer - Disruption of normal flowering times cause lower density forage - Heat snaps cause early exit from winter dormancy	5%

Table 1: Anthropogenic factors which contribute to pollinator decline. [4,32]

Bees have been particularly susceptible to these pressures, with many specialised bee species becoming extinct due to alien species invasion or habitat destruction; ‘colony collapse disorder’ is an enigmatic phenomenon where beehives fail for an unknown reason, emblematic of the sensitive nature of bees [5]. 40% of commercial hives are lost each year due to the aforementioned factors, which also contribute to major causes of bee death, like disease, pests, starvation, and harsh winters [19]. The ‘June gap’ is another phenomenon in the British Isles where the pollen ‘forage’ drops between the heavy spring and summer forages due to suppression of wildflower growth by invasive tall grasses and landscape segmentation, and leads to -beekeepers needing to closely monitor and supplement beehives with food [41].

The purpose of this software is to contribute to the suppression of further pollinator decline; targeted towards amateur beekeepers, automated pollen species identification would provide information on the flowers visited and diet of their bees by honey analysis, enabling effective monitoring of their bees’ health. On a broader scale, this data could be uploaded to an online database, providing valuable information about forages throughout the year, which could be used to investigate the effect of local stressors such as intensive agriculture through a community-based approach.

2.2 Pollen morphology

A biological principle underlying this project is the diversity of pollen morphologies between species and, essentially, the conservation of pollen morphology within a species, as specialised morphological presentation creates specificity to unique pressures of each species’ pollinator mechanism [12]. Light microscopy (LM) is the ‘workhorse’ of palynological study [11], and is commonly accessible to beekeepers. However, of the multitude of pollen morphological features, many can only be reliably observed with electron microscopy, and therefore have been excluded from this study. Observable features are described in table 2 and shown in figure 1, of which terminologies have been standardised as of Punt, *et al.* [29].

Classes	Definition	Number of classes	Examples	Notes
Aperture	A specialized region of the pollen wall that generally differs in ornamentation and/or in structure	10	Colpus, porus, sulcus, inaperturate	Other features such as the organisation and number of apertures can also be defined
Ornamentation	The organisation of features on the surface of the pollen wall	25	Rugulate, psilate, gemmate	Many features are too small to be detected by LM
Polarity	The symmetry of a grain's faces	2	Isopolar, heteropolar	
Shape	2D or 3D descriptor in relation to the equatorial (E) and polar (P) axis	At least 25	Circular, lobate, triangular, oblate, prolate, isodiametric	There are not necessarily clear boundaries in practice; P/E ratio used widely
Dispersal units	Unit in which pollen is naturally dispersed, referred to by number of grains	14	Monad, tetrad planar, tetrad decussate, polyad	Pollen under a microscope are not always in dispersal units
Size	Length of longest axis in μm	Continuous, but 5 suggested classes	Small (10-25 μm), very large (>100 μm); range from 15-200 μm	
Class	Designed grouping of pollen grains with similar features	15	Colpate, plicate, saccate	Crude and non-specific, named after other features, but used widely
Peculiarities /Other	Other recurring features which do not fit other categories	n/a	Ubisch bodies, viscin threads, papilla	Not all are equally/widely described

Table 2: Classes of morphological features of pollen which are used in this study. [10,11]

These features can appear in any combination and, to add to the complexity, 'compound apertures', which are multiple apertures fused together, also occur frequently

[11]. As mentioned in table 2, a feature of morphology is also the organisation and number of apertures which can be defined using prefixes (see glossary).

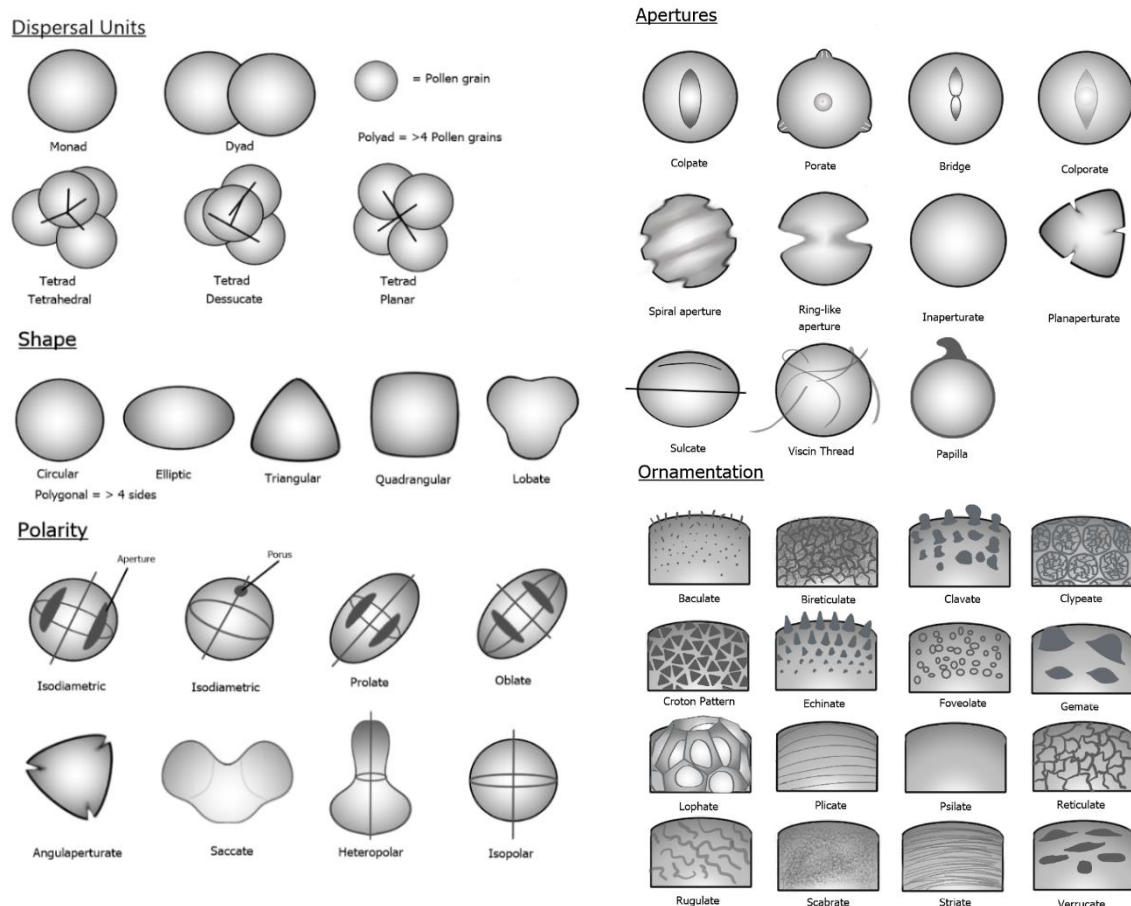


Figure 1: Illustrations of the variety of morphology in pollen. Features which cannot be visualised by LM are not included. For definitions and examples of each term refer to Punt *et al.* [29] Halbritter, *et al.* [11]. Figure created by author using Autodesk SketchBook

2.3 Current research & aims

Automated pollen species identification has long been recognised as a necessity in the palynological community, with wider implications in fields such as ecology, public health, archaeology, and forensics [21]. Gonçalves, *et al.* [8] assessed the accuracy of manual pollen identification by beekeepers, the target of this software, to be 64%, which is too unreliable for effective data collection, not to mention a laborious process. Given this, and the urgency of the aforementioned pressures facing the apiarist community, this project aims to produce a versatile, reliable, and accessible program intended for pragmatic use by beekeepers, aiming to classify morphological features of pollen from LM images and use this

profile to match a database of morphological descriptions, rather than classify each species as a whole, as is the approach in the scholarship.

Prior studies have been primarily proof-of-concept work, using different techniques to classify a limited range of species of pollen, the highest diversity being 46 species [37], employing a dataset of a minimum of 20-30 images per species [26], which is difficult to obtain for a sufficiently diverse database. Other research remains inaccessible in the context of public use, which use specialised microscopes to create 'z-stacks', a series of images taken at different depths of focus to create high-resolution, information-rich data, including those with the highest rates of classification accuracies (in brackets): [2] (96.3-100%), Daood, *et al.* [6] (97.95%), and Sevillano, *et al.* [37] (98%).

While our approach is unique within existing literature, comparable efficacies from similar research in terms of LM use, image format, and development process include Ticay-Ribas, *et al.* [40] (96.4%), Nguyen, *et al.* [26] (92%), Gonçalves, *et al.* [8] (97%), and Khanzhina, *et al.* [15] (95.9%), with the highest diversity being 23 species [8]. It is worth noting that, while several of these are older publications, attempts since 2018 have been dominated by the use of convolutional neural networks (CNN), which will be discussed further on.

3. Methodology

The planned development process is as follows:

1. Gather LM image dataset and database of described pollen morphologies
2. Develop scripts to identify and segment pollen grains from images
3. Integrate functions to measure properties of the grain
4. Use machine learning (ML) to classify these measures into groups based on specific features
5. Automate comparison of the features against the described pollen database

An imperative step in ML, which is a focal point of this report, is refining data to improve the model's robustness; ML provides information about the quality and 'structure' of the data, which in turn can be used to improve data. A review of techniques used and available in each stage are explored below.

3.1 Initial design

Figure 2 presents the design of this software with the target audience as the primary focus, prior to testing and experimentation; future decisions were made in consideration of this framework.

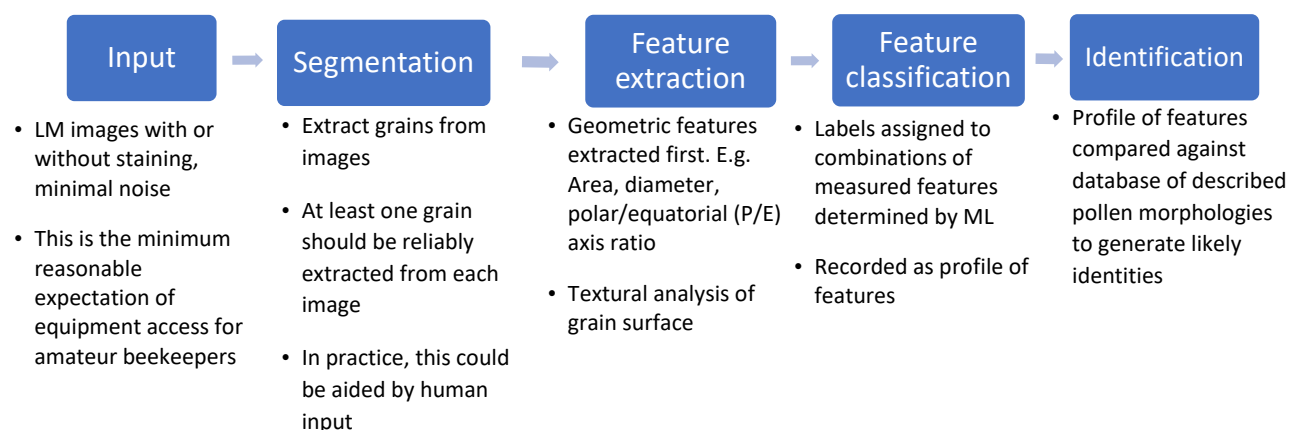


Figure 2: The intended design of PyPollen, created before carrying out the project.

3.2 Choice of techniques/resources

3.2.1 Techniques

3.2.1.1 Datasets

Initially, this project included the collection of samples of honey from beehives around Edinburgh and LM imaging of the samples. Due to COVID-19 restrictions this has not been possible and image datasets have been gathered from online sources. Criteria for an image dataset were decided as: LM images labelled with the species name, >500 images, as taxonomically diverse as possible, and unmanipulated images with simple staining or unstained. Available experimental datasets include:

- POLLEN73S – 2,523 images of 73 species [1]
- POLLEN23E – 805 images of 23 species [36]
- Ranzato, *et al.* (2007) [30] – 1,429 images containing 27 species
- Sevillano, *et al.* (2020) [37] – 19,500 images covering 46 species

While extensive, the diversity is low in all and some use the z-stacks format. Several of these datasets are also already segmented, meaning we cannot test the segmentation features of PyPollen, an essential part of overall functionality. The main drawback is the lack of diversity in these datasets: this software aims to identify the morphological features of any pollen grain and given the vast diversity of morphology possible, the dataset must be as representative as possible. Another option is to use ‘web scraping’ to download images from online palynological databases, of which several are well-established. These include:

- PalDat – The most well-established palynological database with 4,383 species described and most imaged in LM or SEM [10]
- The Global Pollen Project (GPP) – 2,766 ‘digitised reference slides’ of 2,337 species [22]
- Science & Plants for Schools (SAPS) pollen image library – 840 images of 699 species and their sizes (μm) [34]
- Pollen:wiki – German palynological database containing 2,234 species described and 3,483 images [38]

A database of described pollen morphologies is required for dataset annotation for ML, and for comparison of output features. Extensive databases which are consistently described are rare; *PalDat* was chosen because of its status in the palynological community, comprehensiveness, size, and its focus on European species, and will be extracted using web scraping.

3.2.1.2 Segmentation

Automated pollen grain identification using LM images presents challenges due to frequent noise and clustering of grains, as most methods are based on the principle that pollen grains create clear discolouration in LM images. Previous attempts have largely relied on edge detection filters such as Sobel, binary filters, connected snakes, and active contours [26].

A non-filter-based approach is ‘watershed’, used in Bourel, *et al.* [2] & Khanzhina, *et al.* [15]. This utilises the ‘topography’ of the image determined by the minima and maxima of the image, which represent be the highest and lowest greyscale values (lightest and darkest regions) or could be set using values such as hue or saturation. A ‘marker’ is set in the area to be filled and a ‘flooding’ algorithm fills the topographical basin to segment the grain from the image. This is useful when grains are clustered together, as these filled basins do not overlap [18].

3.2.1.3 Feature extraction

This research aims to be able to extract and classify a wide range of pollen features, using standardised Punt, *et al.* [29] terminology as the classes, described in table 2.

Geometrical features were measured with Python package Scikit-learn (see 4.1.2). Popular choices for textural analysis are Gabor wavelet transformations and Fourier fractal descriptors, which use wavelets of defined orientation and magnitude to identify repetitive regions of images [7]. While well-established, they are outdated; more commonly used is grey-level co-occurrence matrices (GLCM). GLCM uses the grey-scale values of an image (0-255) to return a matrix where each pixel represents the change in value relative to its neighbours [24]. Common repetitive motifs can be determined and general properties (see 4.1.2) can be measured [25]. Using ML, these features can be sorted into categories, pre-defined by labels.

3.2.1.4 ML

Unsupervised ML algorithms, such as K-means, use unannotated data to look for structure or ‘relatedness’ and attempts to organise them into a pre-defined number of related ‘clusters’. Supervised ML techniques use annotated data to look for patterns between the labels, building upon the relationships it already understands, in order to classify data into groups. These include algorithms such as support vector machines (SVM), as used in Gonçalves, *et al.* [8] and Mahbod, *et al.* [20], which creates boundaries or ‘hyperplanes’ in n-dimensional space [39]. CNNs are also a complex type of supervised learning algorithm used commonly in image analysis and the discussed literature, which use multiple layers to find hidden patterns between all parts of the image. These layers can be user-defined filters and can therefore be trained to learn and optimise any function, including segmentation and feature classification. Supervised ML models are far more robust and reliable for feature prediction than unsupervised models, but are prone to ‘overfitting’ data, where the model has learned the relationships within the training data too well, and so new data is unreliably classified, and the model cannot ‘generalise’ well. This is a prominent issue within ML, particularly within scientific applications, so steps are taken to avoid this. This will be explained in the results.

Principal component analysis (PCA) is a dimensionality reduction technique, which uses the most significant input parameters to represent the structure of data in one dimension [14]. K-means clustering is underpinned by PCA and is used to assess the quality of our data by prediction models. SVMs are also trained to make predictions in comparison with K-means.

3.2.2 Selection of modules and resources

This program is coded in the Python language, for its comprehensive and well-established applications in data science, as well as its reputation as a ‘learner language’, fitting to the skillset of the researcher. ‘SciPy’ is an open-source library, fundamental for applications in science, maths, and engineering. Sub-packages built on the SciPy framework contain essential functions such as linear algebra, multidimensional image processing, and statistics [43], and are used widely in this software, shown in table 3.

Package	Description	Applications in this project
NumPy (Harris, <i>et al.</i> , 2020)	Provides the basis for most scientific applications of Python. The 'ndarray' is the fundamental object of NumPy, which can represent images, be manipulated easily, and has functional implementation in many other packages	• Images are largely manipulated while in 'ndarray' form • Information about images extracted with NumPy, such as maximum and minimum values
Pandas (McKinney, 2010)	Pandas uses its 'DataFrame' object to easily handle large datasets. It is highly versatile in terms of inputs/outputs and manipulations applicable to DataFrames	• Handling datasets of script outputs such as pollen grain measurements and morphological descriptions • Matplotlib and IPython implementation mediates visual representation of data
Scikit-Image (van der Walt, <i>et al.</i> , 2014)	'Image processing toolbox' for SciPy which provides many image transformations in sub-modules such as 'measure', 'filters', and 'segmentation'	• Provides the backbone for most transformations in this program • Geometric and textural analysis all performed through Scikit-Image • Segmentation mediated by Scikit-Image
Scikit-Learn (Pedregosa, <i>et al.</i> , 2011)	Provides supervised and unsupervised learning algorithm implementation for SciPy	• Most supervised and unsupervised algorithms are supported by Scikit-Learn • PCA and evaluation techniques included
Selenium ("Selenium WebDriver," 2021)	Toolset for web browser function automation	• 'WebDriver' mediates web scraping by directing between pages in a web browser
BeautifulSoup (Richardson, 2020)	Designed to navigate and extract data from HTML files	• HTML files of pollen databases listed above provided by selenium, which BeautifulSoup can extract text and URLs from • Used in all web scraping scripts

Table 3: Python libraries used in the development of this software, their broad functionality and how they are used within this program.

A select few other packages are used throughout the project, such as *difflib* for label matching, *Python Imaging Library (PIL)* for loading and basic editing of images, and *pathlib* for defining file paths.

4. Implementation

Scripts created in this project are available on the 'pollen' repository of the Wells-Wood Research Group github page.

4.1 Data collection

To create an offline pollen morphology description database, *PalDat* was chosen to perform web scraping on. The script for extracting descriptions and images from *PalDat* is explained in Appendix I. After scraping, the data was formatted to provide 13 distinct descriptive labels for each species. Images were not used in further development but serves as an example for image scraping used elsewhere.

The SAPS pollen image library was chosen for its similarity to expected LM images from users. The database also includes grain sizes in μm , useful for validation and scaling techniques. In development of the segmentation functions, it was concluded that some images from the SAPS library contain excessive debris, noise, only have grains not fully in frame, or contain too much clustering and are therefore unsuitable under the limitations of computer vision, as well as reasonable expectations for the quality of images input by users. Examples of suitable versus unsuitable images are shown in shown in figure 3. Later in development, the GPP database was also scraped to improve ML training, examples of which are also shown in figure 3. All experimentation prior to ML was carried out with the SAPS dataset only.

Once databases were downloaded, the species in SAPS were interrogated against *PalDat* using 'difflib' closest matching, which provides tolerance for typos and differential spellings, revealing a cross-over of 331 images (within SAPS; 1,052 when combined) which can be fully annotated.

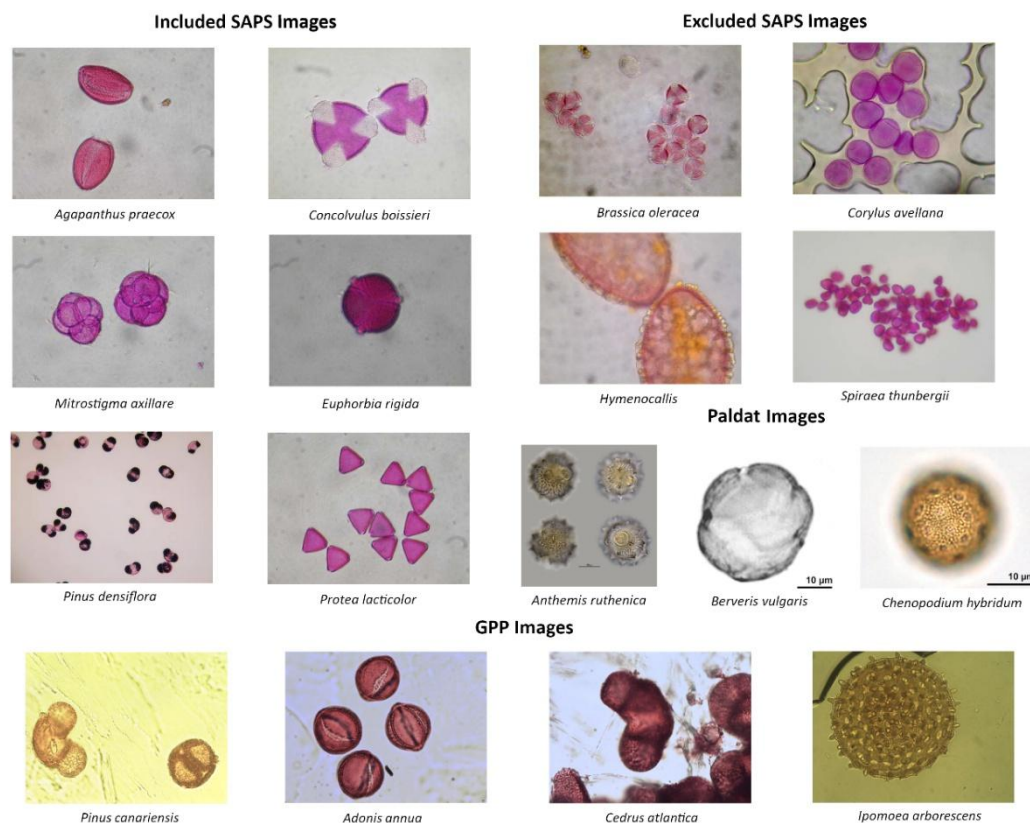
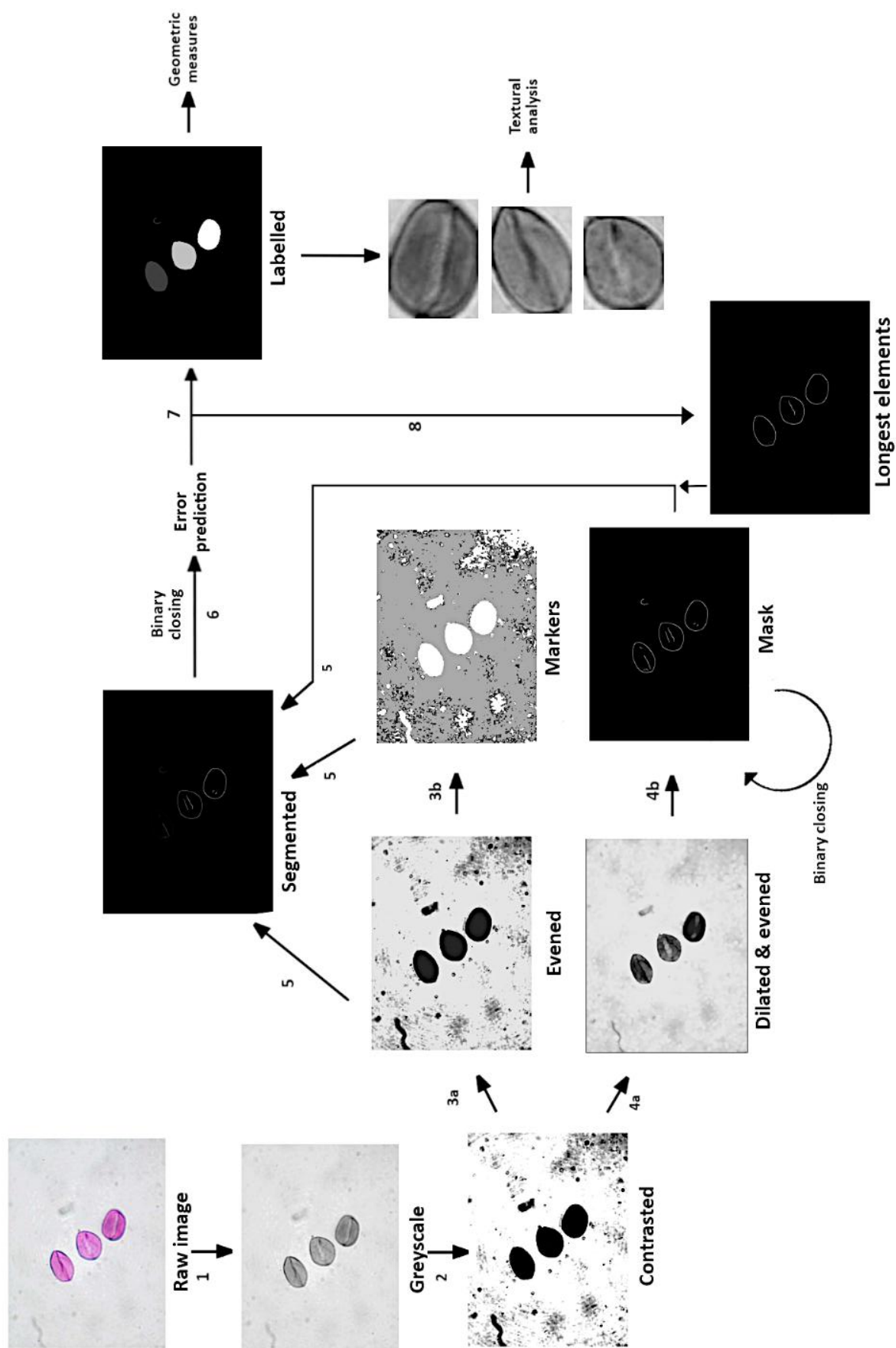


Figure 3: Examples of the dataset used during development of the software. 'Excluded SAPS images' are excluded for the following reasons starting with 'Brassica oleracea' (Clockwise): Too much noise, incorrect experimental preparation, too much clustering and overlapping, no grains fully in image. A sample of images found on the PalDat database is shown. It contains many images, but the style and methods vary greatly, so are not an optimal choice for training and testing this software. Figure created by author in GIMP 2.10.24

4.2 Image segmentation

After experimentation with different edge detection filters and region filling algorithms, the strategy of using a canny filter as a 'mask' and watershed flooding for region filling was settled upon. The canny filter shows exceptional accuracy in edge detection, drawbacks only including the detection of smaller regions and the need to optimise input variables, which can be worked around using binary closing and an optimisation script, respectively. Watershed was chosen because of its versatility when detecting clusters of pollen grains; 'basins' are filled from mid-points and do not overlap or fuse when they meet. However,



1Figure 4: Visualisation of the manipulations made to images processed with the `get_props` function. Figure created by the author in GIMP 2.10.24

issues with identifying the mid-points on grains, especially those in clusters, meant that grey-level extremities were used as filling points for the training of this system.

The segmentation process, shown in figure 4, is as follows:

1. Input images are converted to a greyscale, so each pixel is assigned a grey-level from 0-255 and manipulations in other packages can be applied
2. Contrast is increased to enhance the clarity of the pollen grain edges with the `PIL.Image.inc_contrast` function. Different degrees of contrast increases are required for different functions
3.
 - a. Scikit-Image-Filter's function `rank.mean_bilateral` applies a local average flattening filter, 'smoothing' the image and removing noise
 - b. Markers are used to define where watershed flooding should initiate. The maxima and minima greyscale values are extracted, and markers are set as values within 0.25 standard deviations (SD) below or above respectively. This again works off the principle that grains in the image will cause the greatest fluctuations in brightness.
4.
 - a. `dilation` in Scikit-Image-Morphology shrinks dark areas of the image, which has the effect of making space between pollen grains, allowing the program to distinguish between clustered particles more effectively. This causes negligible loss in data
 - b. The canny edge filter is applied and `binary_closing` joins near 'elements' (lines) together. This 'mask' is applied to the image during watershed to define the regions it should be applied to. This removes larger areas of noise, which are exacerbated by the increased contrast
8. If no grains are detected after step 6, the function is repeated but after the canny filter is applied, the 8,4, or 2 longest elements are identified and binary closing is performed, which is then used as the mask. This is useful when the grain occupies a large proportion of the image, as it avoids smaller internal features which are often detected as grains

5. Watershed is applied and fills the regions of the image containing pollen grains, outputting an array where the edges of separate regions are assigned a unique value. This function works based on connectivity between pixels, which is why increasing contrast and smoothing the image enhances the watershed functions' power. A threshold for grains is applied at this point of area must be $> 1\%$ of the image and a circularity factor set highly, based on the principle that all pollen grains are roughly circular
6. Often these filled regions contain gaps due to extreme texturing or anomalous noise generated by the software, so binary filling is used to create a solid region. Scikit-Image-Measure's `label` function fills these array regions based on the assigned values

Once these regions are labelled, `regionprops` in Scikit-Image-measure provides geometric features including the bounding box – co-ordinates for the rectangular area encasing the pollen grain. This box is defined on the original image and `where` defines any pixel not in the detected grain region as 0, so textural features can be assessed without background interference.

Referring to Appendix II, 11 input variables are required for the `measure_props` function requiring float (decimal) or integer values. To identify the optimal combination of input parameters, `optimise_segmentation_variables` cycles through combinations of values within defined ranges for `get_props` and returns the percentage of detections against actual number of grains, counted as all grains fully in the image by the researcher, and the measures, which can be interpreted to identify the optimal combination of variables. A grain is only counted as a positive result if the major axis length (pixels) multiplied by a conversion factor is within 2 SD of the actual size in μm , thereby avoiding erroneous detection and optimising the function to detect pollen grains only.

4.3 Analysis techniques

Each stage of development requires statistical analysis to ensure the highest efficacy and identify any under-performing stages of the pipeline. Pandas uses integrated Matplotlib functionality to produce figures and statistics. Pandas' `describe` function provides descriptive statistics of the distribution of DataFrames. SciPy also offers statistics functions including Pearson's rank, used in this project.

Scikit-Learn is used for PCA dimensionality reduction, to assess the data quality either in terms of raw input or the efficacy of the code under testing. K-means clustering and SVM prediction is later applied to assess the discriminatory power of our data, estimating how many categories the data can be reliably divided into.

5. Results

5.1 Segmentation

5.1.1. Particle detection

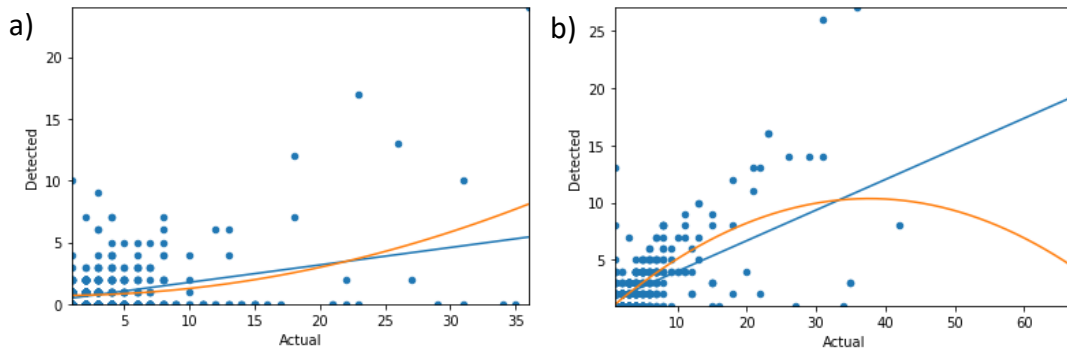


Figure 5: Scatter plots of the no. of detected grains vs the human-validated actual count of grains. a) first attempt, Pearson's rank = 0.345; b): second attempt after addition of selective features and parameter optimisation, Pearson's rank = 0.594. Blue trendline – Linear; Orange trendline - Polynomial

Figure 5 shows the efficacy of grain detection for this software on the reduced SAPS dataset.

Figure 5a shows little correlation, skewed by no grains detected in many images. Changes were made to improve the efficacy including:

- The addition of a circularity threshold to filter debris/noise
- Application of the canny edge filter as a mask for watershed
- The optimisation of input parameters
- Adjustment of percentage calculations in optimisation to account for over-detection
- Re-adjustment of actual count to those at least 80% in the image

A vast increase in positive detection of grains from a Pearson's rank of 0.345 to 0.594, which is considered moderately correlated. The design of this software, while preferable, does not necessitate the detection of every particle in an image.

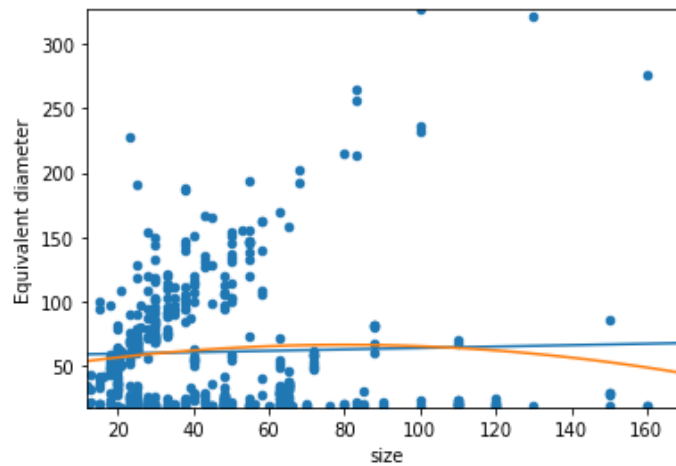


Figure 6: Actual size:detected size correlation, before magnification issues were realised. Pearson's rank = 0.0349. Blue trendline – Linear; Orange trendline - Polynomial

Figure 6 shows the correlation of actual size (μm) against the equivalent diameter in pixels detected by `get_props`. Pearson's rank (0.0349) determines that there is no correlation in this data. However, there is clearly a positive linear trend of some data. There are also many points converging on the X-axis, which could be erroneous, but correlation could also be presumed. It was theorised that the images from the SAPS database were a mixture of images taken at 40x and 100x magnification. This would mean that the diameter in pixels has no single conversion to μm , and at least 2 separate factors would be needed.

Using the parameters determined using only the count accuracy in the optimisation script, the reduced dataset was measured in its entirety and merged with the true sizes taken from the SAPS database. As the variables are not optimised to their full potential, there was significant degree of erroneous detection. To improve the reliability of these calculations, outliers in images with 3 or more grains detected were removed by calculating the absolute z-score and removing those with a score <1.5 . Dividing the major axis length of each pollen grain in pixels by the size in μm , the factor which geometric measures can be multiplied by to return the size in μm can be determined. These calculations, displayed in figure 7, show the distribution of these factors in the dataset. The presence of 2 clear peaks confirms the theory that there are 2 separate magnifications. We can further verify the magnifications being 40x and 100x, as $100/40=2.5$ and the using the means in pixels from table 4, $84.3/33.7=2.5$.

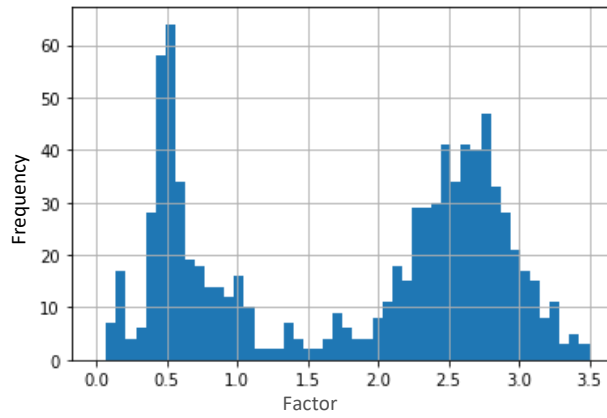


Figure 7: Histogram of detected size to actual size division factor for all particles detected in the reduced dataset

Statistic	40x diameter	Factor	100x diameter	Factor
Count	339	339	517	517
Mean	33.697	0.619	84.256	2.587
SD	16.555	0.283	43.305	0.372
Variance	274.10	0.081	1873.7	0.139
Min	17.877	0.097	19.771	1.521
25%	21.200	0.451	57.159	2.353
50%	28.524	0.543	72.857	2.620
75%	39.774	0.763	95.079	2.826
Max	106.11	1.480	394.60	3.495

Table 4: Descriptive statistics of each distribution seen in figure 7. Generated in pandas.

The distributions were split at $X = 1.5$ and each distribution was statistically described as shown in table 4. While the distributions may appear broad when there should just be 2 values, this can be accounted for by factors such as orientation of the grains, noise interfering with the detected region, normal genetic variations in grain size, and inaccuracies in the true size references. This is accounted for in future calculations by returning the affirmative if the detected value is within 2 SD ($P > 0.95$) of the true size. This is an appropriate range as there was no overlap between 40x and 100x when tested on this dataset.

Once these factors were calculated, both can be applied to the output of the `measure_props`. Plotting the equivalent diameter with the correct factor applied against the true value greatly improves correlation, shown in figure 8.

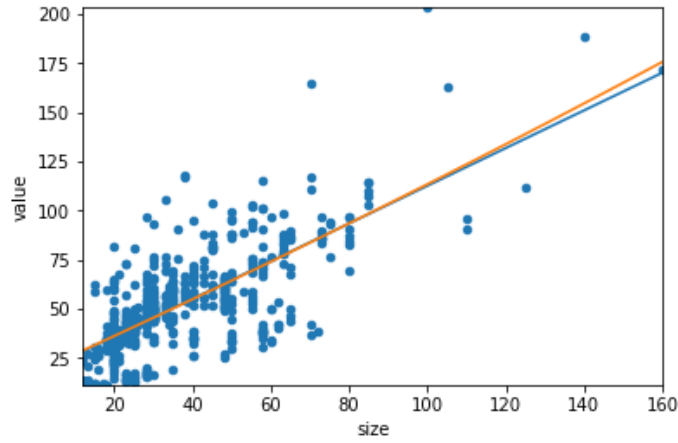


Figure 8: Detected size (μm) against actual size (μm). Pearson's rank = 0.728

A Pearson's rank of 0.728 indicates a strong positive correlation which, when considering the factors influencing wide distribution, is positive. After full development it would be the hope that the user would input the magnification or if unknown, both factors can be applied, and separate probabilities of grain identity can be returned, as true magnification cannot be calculated without the true size (μm) and thereby the species identity.

5.1.2 Feature extraction

15 features were measured from each grain as follows:

- Geometric: Area, Major axis length, Minor axis length, Perimeter, Equivalent diameter, Extent (P/E ratio), Eccentricity
- Textural (GLCM) features: ASM, Contrast, Correlation, Dissimilarity, Energy, Entropy, Homogeneity, Solidity

5.2 Machine Learning & Data polishing

ML requires large quantities of high-quality data, with a minimum of 30 images per feature to effectively learn patterns used here, as per Nguyen, *et al.* [26] and other attempts. More images will be needed when classifying more complex and minute features with consideration of factors such as poor-quality images, different preparation techniques, different orientations, and compounded features. The SAPS dataset does not provide enough data when classifying 13 features with up to 25 classes in each, and so the GPP

database was also accessed for ML applications. Due to differing resolutions between datasets, geometric features were represented as a percentage of the total image area, and magnification factors were adjusted with an image height ratio, therefore requiring images to be in standard 4:3 aspect ratio. This combined dataset is used hereafter.

The Adjusted Rand Index (ARI) is used widely to assess ML models and is a measure of similarity between the predicted classes and true values adjusted for chance correct labelling and so represents the proportion of correct predictions the model makes while most commonly applied in supervised ML it can provide comparative quantitative assessment on all following techniques [33].

5.2.1 PCA

The fully measured SAPS dataset returned 2,130 detected. Prior to adjustments for magnification, PCA was applied and visualised in 2 dimensions to identify any potential discriminatory power, as seen in figure 9a.

While the segmentation optimisation script detected 72.3% of all grains in the 20-image sample accurately, when applied to the whole dataset, this decreased to 45.7%. As the size cross-validation was only applied during optimisation, 44.3% of detections made were erroneous. In order to determine structure within the non-erroneous measurements, PCA was applied of the dataset with errors marked and removed, and magnification adjustments applied.

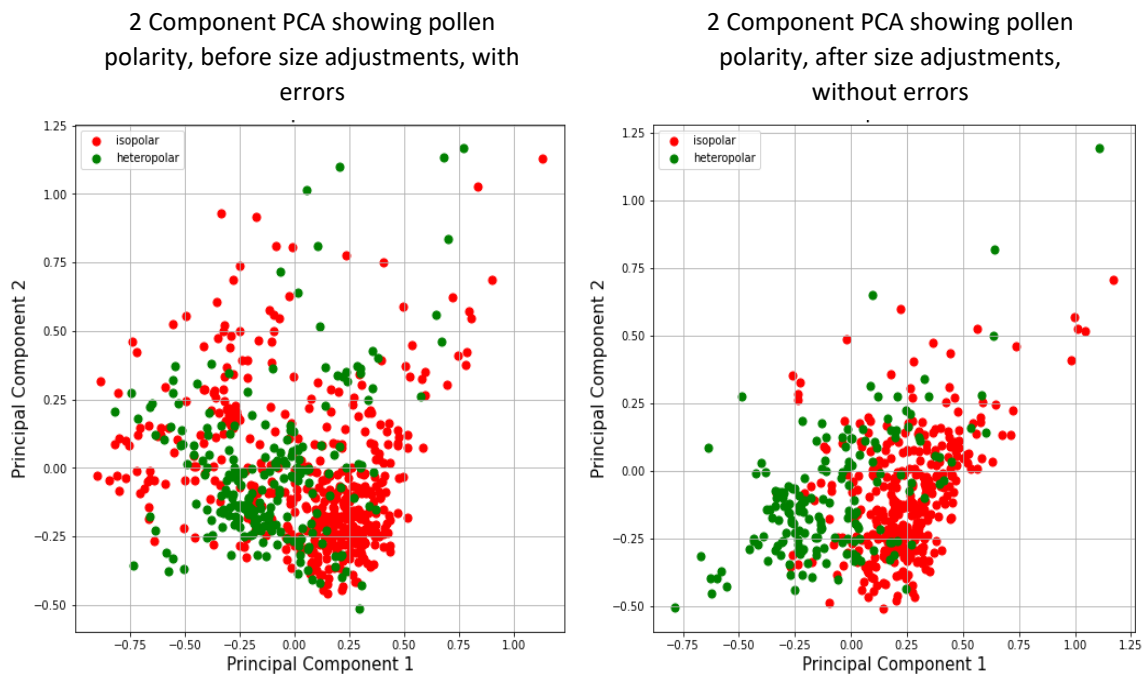


Figure 9: PCA with 2 components. a) reduced set and b) reduced set with errors removed and size adjustments. While PCA can identify more structure with more principal components, they cannot be visualised, so the 2 most representative components are displayed. The labels 'isopolar' vs 'heteropolar' are selected as examples, as they should be significantly morphologically different and interpretable through many of the measurements, so should be easily distinguishable. Left - ARI = 0.0; Right - ARI = 0.352

2-component PCA showing high degrees of structure should show clear groupings of labelled datapoints. Figure 9a exhibits vast overlapping and therefore no structure. Figure 9b shows improvements, but there is still too much overlap to indicate strong enough discriminatory power between these features, and improvements must be made to the data.

5.2.2 K-means clustering

Given these results, it was planned that ML could be used to categorise errors from hits (non-errors) and be implemented into the segmentation code. Using the combined dataset with labelled errors, K-means looks for patterns to categorise our data into clusters, to assess data's structure and predictive capabilities. This can be visualised in figure 10, which shows a fairly strong difference between errors and hits.

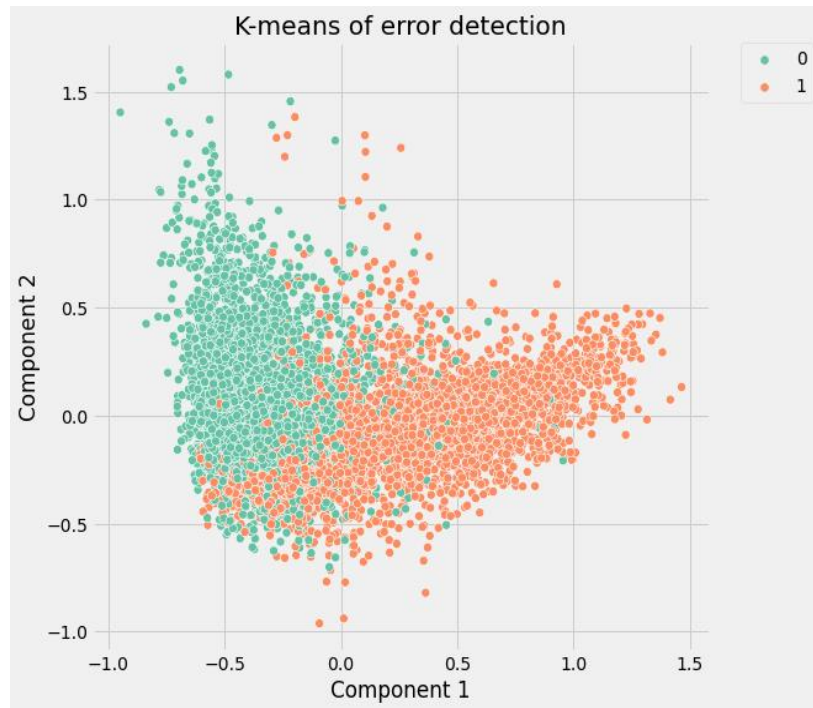


Figure 20: K-means clustering on data fit to a 2-component PCA model, labelled with 1 = errors vs 0 = non-errors (hits). ARI = 0.473

This clustering is strong enough to perform supervised learning in order to classify errors; using 6 components the clustering performed better, returning an ARI of 0.624

4.2.3 Support Vector Machine (SVM) supervised learning

SVMs work to classify labelled data in n-dimensional space by finding patterns between the data and creating boundaries. SVM models were trained using defined errors, the results of which are shown in table 5. Despite the lack of structure seen, the efficacy of feature classification was also assessed. Variability is common in ML depending on the input data, as well as the aforementioned overfitting. In order to improve reliability, K-Folds cross-validation was used, in which the model is 'fit' (trained) 10 times, and each time a different 10% of the data is used for testing and 90% is used for training, and the mean is taken. Performance was also corroborated with a 'balanced accuracy' measure (not shown). Also, in line with prior structure, any class with fewer than 30 datapoints is not included. It is optimal to have equal weightings of classes within a dataset, and so the size of the annotated dataset and the standard error of the mean (SEM) has also been included to demonstrate the quality and balancing of the training set [27].

Feature	Measure	SAPS		GPP		Combined	
Classifier		SVM	K-M	SVM	K-M	SVM	K-M
Errors	API	0.684	0.576	0.88	0.402	0.817	0.40
	Dataset size	2130		7712		9842	
	SEM(Classes)	2.622 (2)		7.608 (2)		2.555 (2)	
	Dataset size	944		4231		5175	
	SEM(Classes)	9.406 (2)		28.904 (2)		30.155 (2)	
P/E	API	0.208	0.041	0.140	0.0335	0.183	0.0614
	Dataset size	256		475		731	
	SEM(Classes)	3.523 (3)		0.788 (3)		2.675 (3)	
Phyla	API	0.731	0.0027	n/a		0.479	0.0623
	Dataset size	527				1659	
	SEM(Classes)	9.823 (2)				19.089 (2)	
Polarity	API	0.437	0.023	0.274	0.243	0.471	0.161
	Dataset size	511		1090		1601	
	SEM(Classes)	4.357 (2)		12.873 (2)		13.0834 (2)	
Family	API	0.663	0.451	0.189	0.0853	0.228	0.0597
	Dataset size	154		499		941	
	SEM(Classes)	0.367 (4)		0.761 (10)		0.878 (17)	
Order	API	0.413	0.162	0.115	0.054	0.127	0.035
	Dataset size	271		916		1458	
	SEM(Classes)	0.397 (6)		1.079 (14)		1.14 (20)	
2D shape	API	0.337	0.0459	0.201	0.0843	0.354	0.0829
	Dataset size	491		1117		1608	
	SEM(Classes)	6.433 (3)		13.047 (3)		14.398 (3)	
Class	API	0.332	0.174	0.112	0.0299	0.178	0.0318
	Dataset size	481		1005		1564	
	SEM(Classes)	3.343 (5)		5.868 (4)		6.328 (7)	
Ornamentation	API						
	Dataset size						
	SEM(Classes)						
Aperture prefixes	API						
	Dataset size						
	SEM(Classes)						
Aperture type	API	0.295	0.0744	0.103	0.0242	0.178	0.0296
	Dataset size	494		1040		1582	
	SEM(Classes)	4.47 (5)		6.0877 (4)		6.82 (6)	
Aperture features	API	n/a		0.114	0.0107	0.206	
	Dataset size			391		569	
	SEM(Classes)						
Dispersal unit	API	n/a		0.247	0.0433	0.143	0.0421
	Dataset size			1133		1663	
	SEM(Classes)			15.67139 (2)		19.237 (2)	

Table 5: Performance of SVM and K-Means models on classifying morphological features of pollen as well as errors from accurate hits and different types of errors from hits, as well as against each other. The dataset size (limited by annotation),

standard error of the mean (SEM), and number of classes within each dataset. A low SEM represents an equally balanced dataset.

As the SVM model can detect errors with an ARI of 0.817, the efficacy of the segmentation function is increased to ~81% true values. With detection this reliable, the parameters for segmentation can be re-optimised and the filters removed to increase the proportion of grains extracted from each image.

Specific features cannot be distinguished with any reasonable reliability, with no model achieving an ARI higher than 0.471 (polarity, 2 classes). Distinction between taxa such as phyla was somewhat accurate using the SVM (0.731, SAPS). However, the K-Means model achieved an ARI of 0.0027, representative of SVM overfitting due to an imbalanced dataset. The frequency of gymnosperms was far lower than angiosperms (SEM = 9.838), meaning that, due to how SVMs function, it is likely to have grouped the gymnosperms together and predicted that anything else is an angiosperm, which in practice, is invalid. This is the primary issue faced with supervised learning models, also observed with 2D shape (all) and polarity (Combined).

6. Discussion

The aim to create software which can read LM images, automatically identify pollen grains within them, extract the region from the image, perform measurements on the grain, classify morphological features, and predict species identity is an ambitious one. As discussed, many attempts have sought to produce reliable prediction models, but most focus on proof-of-concept technical approaches and are too limited in species diversity to create an effective tool in combatting pollinator decline. This research presents the groundwork for versatile, accessible, and pragmatic software for public use.

6.1 Performance

The unique approach of this research and the stage at which this project ended make it challenging to position it within the scholarship. Segmentation has been achieved in other research through specialised microscopes which identify the grain before the image is taken or by a CNN which does not function in a comparable way. Nguyen, *et al.* [26] used a similar watershed segmentation, reporting an 89.5% accuracy and 93.8% recall. While the segmentation software initially underperformed, the implementation of the SVM has increased accuracy to 81.7%

Feature classification returned very poor accuracies. It could be argued that classifying features is more difficult than species as features have fewer significant measures which can be interpreted, but the classification of features remains in desperate need of improvement. Overfitting caused issues in multiple features, which is why it is imperative we use techniques such as K-Folds and assess the dataset quality to give a comprehensive and reliable evaluation of our models. While an SVM is expected to perform better with classification than K-Means, a difference in API of such magnitudes as seen in the above examples is unlikely. Instances of successfully trained SVMs can be seen with errors (Combined) and family (SAPS).

Success in classification by taxonomic rank, as within previous attempts, but not in classification of features, provides evidence that our approach is more complex, requiring a

higher quality and quantity of data to achieve equal fidelities, and as such is difficult to compare against existing literature. MLL has been effectively applied in this research, but more data, exhaustive testing, and sophisticated ML models are required to further PyPollen's capabilities

6.2 Critique

It is important to evaluate the development of this project in the context of its intent. In application, human input for magnification, location, seasonal time, and scale could contribute to classification of grains, and even the segmentation could be augmented by human provision of the co-ordinates to initiate watershed flooding from or through 'human-in-the-loop' ML, which optimises a function through learning human input [9]. This research works to train the model on which an application would be based, and thereby necessitates the more complex automation of processes like segmentation, annotation, and determination of magnification, in order to process large training datasets, while real-world implementation would be likely to have greater success.

Data collection occupied a significant proportion of the time allocated for this project. We realised early on that the availability of large amounts of data, which met the criteria to ensure our program fit the intended use, was relatively low, particularly for morphological descriptions. Compounded by the intricate and specific nature of web scraping, which had to be learned, time for development of the identification program was limited. The lack of morphological descriptions was problematic because although the combined dataset contained 4,667 detected grains, the dataset was restricted to 1,666 due to lack of annotations, arguably still insufficient for effective ML.

Compounding of the GPP dataset at an earlier point would have advanced the stage of ML achieved; a large proportion of time was spent trying to improve the SAPS data quality and performing inconsequential ML when the sample size was too low, but GPP was thought inaccessible until near the end. The quality of each dataset for ML is assessed in table 5. The SAPS dataset generally saw better and more reliable predictions than GPP, although also often had fewer classes to distinguish between. Issues with quality were present in the GPP dataset, although even if removed, it is unlikely that the performance of

classification would have improved significantly. Regardless, the quality of data in large quantities was a limiting factor in this research.

The segmentation functions took most of the development time. The variety in slide preparation, size within the image, clustering of grains, and orientations made it difficult to generalise the script, especially when presented with the new GPP images, as suggested by the increase in errors from SAPS (44.3%) to GPP (60.0%), although this will also have been due to the optimisation of parameters trained on images from SAPS.

In order to improve segmentation as well as classification, CNNs should have been used but time constraints prevented this. This represents the most significant change to strategy that could have been made. As mentioned, CNNs can be trained to any function, including the segmentation of pollen grains from images, performed successfully in Bourel *et al.* [2] and Mahbod, *et al.* [20]. In order to be competitive with current literature in any image-based research, CNN use has almost become a necessity, as it is unparalleled in its ability to identify hidden relationships between images. While also prone to overfitting, this can be reduced with fine-tuning, which would have been viable to show some classification of features using this dataset, but require >1,000 images per class for optimal efficacy [27].

While the gathering and annotation of a dataset, and the development of segmentation functions which utilises ML was successful, the aims of this project were ambitious, with the degree of classification required greater than expected, and as such, the quality of data and sophistication of ML models used were limiting.

6.3 Future steps

As a result of this research, the framework for further development has been well established and steps following the same strategy can be taken. 2 primary steps can be taken to improve classification with the SVM: the introduction of more features, and the addition of more diverse data within feature classes. There is room for expansion of textural analysis; the current GLCM features measure overall properties of the grain's texture, but the identification of repeating 'motifs', representative of minute features which are abundant in pollen morphology such as apertures and ornamentation, could prove particularly valuable. Techniques such as Gabor wavelets or Haar-like features perform similar measurements to GLCM but segment the image into smaller areas and compare

these areas to assess differential texture. There are also further GLCM manipulations which have been used for biological applications which could be made that, for example, look at 3D structure, multi-level (of smoothing), and directional analysis as discussed in Nanni, *et al.* [25]. Data from the hue and saturation channels have also not been exploited here, as they require more intense pattern recognition when using a diverse dataset, but has been effectively used in Gonçalves, *et al.* (2016) and those which utilise CNNs. These could also be used concurrently with a CNN, which again, would improve classification greatly.

While the current dataset is large, many classes have not been considered in the model training because there are fewer than 30 examples. If more images were sourced, ideally through the creation of our own from local hives, classes with greater values could be reduced to create a more balanced dataset, and therefore a more reliable ML model. Even with a small dataset for each class, 'data augmentation' can be used to improve the dataset for CNN training, where images are manipulated (e.g. rotated, mirrored, increased saturation) to create more images to train from.

7. Conclusion

This research presents a pragmatic approach to automated pollen classification which could be widely utilised in supressing further pollinator decline. Literature in this arena has repeatedly proven that LM images can be used for effective classification, but there has yet to be a significant attempt to implement this in a practical way. The gathering and annotation of a suitable dataset, the development of a robust segmentation program using ML for error prediction, and preliminary training of ML models for feature classifications, as well as practical suggestions of steps to achieve the set aims provides a valuable basis for further development of PyPollen. This study and the relevant literature show promise that not only is this approach technically possible, it is practical and a necessity in combatting pollinator decline.

Bibliography

1. Astolfi, G., Barbosa Gonçalves, A., Vilharva Menezes, G., Silveira Brito Borges, F., Christina Melo Nunes Astolfi, A., Takashi Matsubara, E., et al. (2020). POLLEN73S: An image dataset for pollen grains classification. *Ecological Informatics*, 60. doi:10.1016/j.ecoinf.2020.101165
2. Bourel, B., Marchant, R., de Garidel-Thoron, T., Tetard, M., Barboni, D., Gally, Y., et al. (2020). Automated recognition by multiple convolutional neural networks of modern, fossil, intact and damaged pollen grains. *Computers & Geosciences*, 140. doi:10.1016/j.cageo.2020.104498
3. Bradbear, N. (2009). *Bees and their role in forest livelihoods*. Rome, Italy: Food and Agriculture Organization of the United Nations.
4. Cariveau, D. P., & Winfree, R. (2015). Causes of variation in wild bee responses to anthropogenic drivers. *Current Opinion in Insect Science*, 10, 104-109. doi:10.1016/j.cois.2015.05.004
5. Dainat, B., van Engelsdorp, D., & Neumann, P. (2012). Colony collapse disorder in Europe. *Environmental Microbiology Reports*, 4(1), 123-125. doi:10.1111/j.1758-2229.2011.00312.x
6. Daood, A., Ribeiro, E., & Bush, M. (2018). *Sequential Recognition of Pollen Grain Z-Stacks by Combining CNN and RNN*. Paper presented at the International Florida Artificial Intelligence Research Society Conference, Melbourne, FL.
7. Florindo, J.B., & Bruno, O.M. (2011). *Fourier Fractal Descriptors for Colored Texture Analysis*. Paper presented at the Advanced Concepts for Intelligent Vision Systems.
8. Gonçalves, A. B., Souza, J. S., Silva, G. G. d., Cereda, M. P., Pott, A., Naka, M. H., et al. (2016). Feature Extraction and Machine Learning for the Classification of Brazilian Savannah Pollen Grains. *PLOS One*, 11(6), e0157044. doi:10.1371/journal.pone.0157044
9. Grosund, T., & Aanestad, M. (2020). Augmenting the Algorithm: Emerging human-in-the-loop work configurations. *he Journal of Strategic Information Systems*, 29(2).
10. Halbritter, H. (2005). PalDat - A palynological database. Retrieved from www.palдат.org
11. Halbritter, H., Ulrich, S., Grímsson, F., Weber, M., Zetter, R., Hesse, M., et al. (2018). *Illustrated Pollen Terminology* (2nd ed.): Springer Open Access.
12. Hao, K., Tian, Z.-X., Wang, Z.-C., & Huang, S.-Q. (2020). Pollen grain size associated with pollinator feeding strategy. *Proceedings of The Royal Society*, 287(20201191). doi:10.1098/rspb.2020.1191
13. Harris, C. R., Millman, K. J., & van der Walt, S. J. (2020). Array programming with NumPy. *Nature*, 585, 357–362. doi:10.1038/s41586-020-2649-2
14. Jolliffe, I.T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065).
15. Khanzhina, N., Putin, E., Filchenkov, A., & Zamyatina, E. (2018). *Pollen Grain Recognition Using Convolutional Neural Network*. Paper presented at the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, Bruges.
16. Kleijn, D., Winfree, R., Bartomeus, I., Carvalheiro, L. S. G., Henry, M., Isaacs, R., et al. (2015). Delivery of crop pollination services is an insufficient argument for wild pollinator conservation. *Nature Communications*, 6. doi:10.1038/ncomms8414
17. Klein, A.-M., Vaissie`re, B. E., Cane, J. H., Steffan-Dewenter, I., Cunningham, S. A., Kremen, C., et al. (2006). Importance of pollinators in changing landscapes for world crops. *Proceedings of The Royal Society*, 274, 303-313.
18. Kornilov, A. S., & Safonov, I. V. (2018). An Overview of Watershed Algorithm Implementations in Open Source Libraries. *Journal of Imaging*, 4(10). doi:10.3390/jimaging4100123

19. Kulhanek, K., Steinhauer, N., Rennich, K., Caron, D. M., Sagili, R. R., Pettis, J. S., et al. (2017). A national survey of managed honey bee 2015–2016 annual colony losses in the USA. *Journal of Apicultural Research*, 56(4), 328–340. doi:10.1080/00218839.2017.1344496
20. Mahbod, A., Schaefer, G., Ecker, R., & Ellinger, I. (2020). *Pollen Grain Microscopic Image Classification Using an Ensemble of Fine-Tuned Deep Convolutional Neural Networks*. Medical University of Vienna.
21. Maiti, R., Gonzalez-Rodriguez, H., & Raj Ojha, E. (2016). Pollen biology and plant productivity: A review. In *Autoecology and Ecophysiology of Woody Shrubs and Trees: Concepts and Applications* (First ed., pp. 133–157): John Wiley & Sons, Ltd.
22. Martin, A. C., & Harvey, W. J. (2017). The Global Pollen Project: a new tool for pollen identification and the dissemination of physical reference collections. *Methods in Ecology and Evolution*, 8, 892–897. doi:10.1111/2041-210X.12752
23. McKinney, W. (2010). *Data structures for statistical computing in python*. Paper presented at the Proceedings of the 9th Python in Science Conference.
24. Mohanaiah, P., Sathyanarayana, P., & GuruKumar, L. (2013). Image Texture Feature Extraction Using GLCM Approach. *International Journal of Scientific and Research Publications*, 3(5), 290–294.
25. Nanni, L., Brahnamm, S., Ghidoni, S., Menegatti, E., & Barrier, T. (2013). Different Approaches for Extracting Information from the Co-Occurrence Matrix. *PLOS One*, 8(12). doi:10.1371/journal.pone.0083554
26. Nguyen, N. R., Donalson-Matasci, M., & Shin, M. C. (2013). *Improving Pollen Classification with Less Training Effort*. Paper presented at the IEEE Workshop on Applications of Computer Vision (WACV), Clearwater Beach, FL.
27. Nichols, J. A., Chan, H. W. H., & Baker, M. A. B. (2019). Machine Learning: Applications of artificial intelligence to imaging and diagnosis. *Biophysical reviews*, 11(1), 111–118. doi: 10.1007/s12551-018-0449-9
28. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830.
29. Punt, W., Hoen, P. P., Blackmore, S., Nilsson, S., & Le Thomas, A. (2007). Glossary of pollen and spore terminology. *Review of Paleobotany & Palynology*, 143, 1–81. doi:10.1016/j.revpalbo.2006.06.008
30. Ranzato, M., Taylor, P. E., House, J. M., Flagan, R. C., LeCun, Y., & Perona, P. (2007). Automatic recognition of biological particles in microscopic images. *Pattern Recognition Letters*, 28, 31–39. doi:10.1016/j.patrec.2006.06.010
31. Richardson, L. (2020). Beautiful Soup. Retrieved from <https://www.crummy.com/software/BeautifulSoup/bs4/doc/#>
32. Sánchez-Bayo, F., & Wyckhuys, K. A. G. (2019). Worldwide decline of the entomofauna: A review of its drivers. *Biological Conservation*, 232, 8–27. doi:10.1016/j.biocon.2019.01.020
33. Santos, J. M., & Embrechts, M. (2009). *On the Use of the Adjusted Rand Index as a Metric for Evaluating Supervised Classification*. Paper presented at the International Conference on Artificial Neural Networks.
34. Schools, S. P. f. (2011). Pollen Image Library. Retrieved from <http://www.saps.org.uk/pollen/index2.htm>
35. . Selenium WebDriver. (2021). Retrieved from <https://www.selenium.dev/>
36. Sevillano, V., & Aznarte, J. L. (2018). Improving classification of pollen grain images of the POLEN23E dataset through three different applications of deep learning convolutional neural networks. *PLOS One*, 13(9), e0201807. doi:10.1371/journal.pone.0201807
37. Sevillano, V., Holt, K., & Aznarte, J. L. (2020). Precise automatic classification of 46 different pollen types with convolutional neural networks. *PLOS One*, 15(6), e0229751. doi:doi.org/10.1371/journal.pone.0229751

38. Stebler, T. (2011, 11 April 2021). Pollenatlas. Retrieved from <https://pollen.tstebler.ch/MediaWiki/index.php?title=Pollenatlas&oldid=47224>
39. Suthaharan, S. (2016). Support Vector Machine. In *Machine Learning Models and Algorithms for Big Data Classification Based on Geometrical, Descriptors and Color Features Using Decorrelation Stretching Method*. Paper presented at then IFIP Advances in Information and Communication Technology, Corfu.
40. Ticay-Ribas, J., del Pozo-Baños, M., Travieso, C., Arroyo-Hernández, J., Pérez, S., Alonso, J., et al. (2011). *Pollen Classification Based on Geometrical, Descriptors and Colour Features Using Decorrelation Stretching Method*. Paper presented at the IFIP Advances in Information and Communication Technology, Corfu.
41. Timberlake, T. P., Vaughan, I. P., & Memmott, J. (2019). Phenology of farmland floral resources reveals seasonal gaps in nectar availability for bumblebees. *Journal of Applied Ecology*, 56(7), 1585-1596. doi:10.1111/1365-2664.13403
42. van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., et al. (2014). scikit-image: Image processing in Python. *PeerJ*, 2(e453). doi:10.7717/peerj.453
43. Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17(3), 261-272. doi:10.1038/s41592-019-0686-2
44. Willmer, P. (2011). *Pollination and Floral Ecology*. Woodstock, Oxfordshire: Princeton University Press.
45. Yamashita, R., Nishio, M., Kihn Gian do, R. & Togashi, K. (2018). Convolutional neural networks: an overview and application in radiology. *Insights into Imaging*, 9, 611-629. doi: 10.1007/s13244-018-0639-9

Appendix I – *PalDat* web scraping script

1. The Selenium 'WebDriver' was defined to use the Google Chrome browser

```
driver = webdriver.Chrome(  
    executable_path=r"C:/Users/pinto/OneDrive - University of Edinburgh/coding/chromedriver")
```

2. An example page was loaded and HTML elements (lines) with the 'span' tag and 'diag-label' class were identified and the information extracted using BeautifulSoup, which contained the included category titles

```
driver.get("https://www.paldat.org/pub/Abelia_grandiflora/305286")  
content = driver.page_source  
paldat = BeautifulSoup(content, features="lxml")  
catlist = ["Genus", "Species"]  
for c in paldat.find_all("span", class_="diag-label"):  
    ca = c.extract()  
    cat = ca.string.extract()  
    catlist.append(cat)  
with open("species.csv", "w", newline="") as out:  
    out = csv.DictWriter(out, catlist)  
    out.writeheader()
```

3. A dictionary object was defined to hold image URLs and a 'for' loop was set to access each page containing genus names by letter

```
images_url = {}  
for L in ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z"]:  
    driver.get(f"https://www.paldat.org/search/{L}")
```

4. Elements with the 'genus' class are extracted, and WebDriver directed to each genus' page

```
genus = driver.find_elements_by_class_name("genus")  
glist = []  
for a in genus:  
    glist.append(str(a.text))  
for g in glist:  
    driver.get(f"https://www.paldat.org/search/genus/{g}")
```

5. To create compatibility between Selenium, which controls the browser, and BeautifulSoup, which 'parses' or accesses the HTML contents, these 2 lines have to be redefined on every page where information is extracted

```
content = driver.page_source  
paldat = BeautifulSoup(content, features="lxml")
```

6. LM image URLs and respective species name for all species in the genus are identified and appended to the dictionary

```
for a in paldat.find_all("td", attrs={"data-grid-property": "LM"}):  
    for b in a.find_all("a"):  
        name = b.get("data-title")
```

```

        images_url.setdefault(name, [])
        url = b.get("href")
        images_url[name].append(url)
    else:
        driver.back()

```

7. Species' names are identified in the genus page and WebDriver directed to the species page containing the morphological descriptions

```

species = driver.find_elements_by_class_name("species")
slist = []
for a in species:
    slist.append(str(a.text).replace(" ", "_"))
for s in slist:
    driver.get(f"https://www.paldat.org/pub/{s}")
    datlist = []
    content = driver.page_source
    paldat = BeautifulSoup(content, features="lxml")

```

8. The species name, taxonomy, and morphological descriptions are all located and extracted using the 'species' class, 'p' tag, and 'diag-value' class respectively

```

for c in paldat.find_all("h1", class_="species"):
    d = c.extract()
    name = d.string.extract()
    name = name.split()
    datlist.extend(name)
    tax_list = []
    for t in paldat.find("p"):
        tax_list.append(t)
    taxonomy = tax_list[1].replace("\n", "").split(",")
    for v in paldat.find_all("span", class_="diag-value"):
        va = v.extract()
        val = va.string.extract()
        datlist.append(val)
    datlist.append(taxonomy)

```

9. The species and its descriptions are appended to a .csv file

```

with open("species.csv", "a", newline="") as out:
    out = csv.writer(out, catlist)
    out.writerow(datlist)

```

10. A file for LM images to be saved in is created if it does not already exist

```

if not Path("paldat_images").exists():
    Path("paldat_images").mkdir()

```

11. The image URLs saved to the dictionary are accessed using 'requests' and saved as a .jpeg with the species name as the filename

```

for key, values in images_url.items():
    for val in values:
        image = requests.get(f"https://www.paldat.org{val}").content
        image_file = io.BytesIO(image)
        image = Image.open(image_file).convert('RGB')
        path = Path("paldat_images", f"{key}")

```

```
image.save(path, 'JPEG', quality = 95)  
image.close()
```

Appendix II – ‘PyPollen’ code

```
"""Module for segmenting and measuring pollen particles from images."""

from pathlib import Path
import typing as t
import skimage
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import skimage.filters as sfi
import skimage.measure as sme
import skimage.morphology as smo
import skimage.segmentation
from PIL import Image, ImageEnhance
from scipy import ndimage as nd
from scipy import stats
from skimage.feature import canny
from skimage.feature import texture as sft
from skimage.io import imread
from skimage.util import img_as_ubyte
import pickle

# Function to increase contrast with PIL function, used throughout other functions
def inc_contrast(
    image_array: np.ndarray,
    contrast_times: float = 15,
    dilation: bool = False,
    dilation_selem_size: int = 10,
) -> np.ndarray:
    """Increases contrast of image."""
    # Increase contrast of input to clarify edges
    enhance = ImageEnhance.Contrast(image_array)
    cont = enhance.enhance(contrast_times)
    cont = np.array(cont)

    if dilation == True:
        # Dilation of image - This shrinks pollen silhouettes to help with clustering
        selem = smo.disk(dilation_selem_size)
        dilated = smo.dilation(cont, selem)
        return dilated
    else:
        return cont

# Function to retrieve maxima and minima of greyscale images for use as markers in watershed flood region filling
def get_markers(
```

```

    image_array: np.ndarray,
    evened_selem_size: int = 4,
    markers_contrast_times: float = 15,
    markers_sd: float = 0.25,
) -> np.ndarray:
    """Finds the highest and lowest grey scale values for image flooding."""
    selem = smo.disk(evened_selem_size)
    evened = sfi.rank.mean_bilateral(
        inc_contrast(image_array, contrast_times=markers_contrast_times), selem
    )
    # Markers defined by highest and lowest grey levels set as markers
    high = np.max(evened)
    low = np.min(evened)
    std = np.std(evened)
    neatararray = np.array(image_array)
    markers: np.ndarray = np.zeros_like(neatararray)
    # Level reduced/decreased by 1/4 SD
    markers[evened < low + (markers_sd * std)] = 3
    markers[evened > high - (markers_sd * std)] = 2
    return markers

# Function to identify grains in an images and extract these regions of the image
def segment(
    image_array: np.ndarray,
    sigma: float = 3,
    canny_lt: float = 25,
    canny_ht: float = 5,
    evened_contrast_times: float = 1,
    dilated_contrast_times: float = 8,
    dilated_evened_selem_size: int = 9,
    evened_selem_size: int = 4,
    dilation_selem_size: int = 4,
    markers_meaned_selem_size: int = 4,
    markers_contrast_times: float = 30,
    markers_sd: float = 0.25,
    return_mask: bool = False,
    use_longest_elements: bool = False,
    iterations=1,
):
    # Evened filter smoothes image to remove noise;
    # selem (Structuring element) flattens local areas within this element
    selem = smo.disk(evened_selem_size)
    evened = sfi.rank.mean_bilateral(
        inc_contrast(image_array, contrast_times=evened_contrast_times), selem
    )
    selem = smo.disk(dilated_evened_selem_size)
    # Dilation also applied for use in edge detection - Shrinks dark areas, pulling clustered grains a
    part from each other

```

```

devened = sfi.rank.mean_bilateral(
    inc_contrast(
        image_array,
        contrast_times=dilated_contrast_times,
        dilation=True,
        dilation_selem_size=dilation_selem_size,
    ),
    selem,
)
# Canny edge detection filter detects object edges and outputs boolean array
elevation_map = canny(
    devened, sigma=sigma, low_threshold=canny_lt, high_threshold=canny_ht
)
# 3x3 structuring element so binary closing detects true values in any neighbouring pixel as a connection
s = [[1, 1, 1], [1, 1, 1], [1, 1, 1]]
if use_longest_elements == False:
    # Binary closing of elements in canny edge detection
    elevation_map = nd.binary_closing(elevation_map, s, iterations=2)
if use_longest_elements == True:
    elevation_map = nd.binary_closing(elevation_map, s, iterations=iterations)
    label_im, nb_labels = nd.label(elevation_map, s)

    sizes = nd.sum(image_array, label_im, range(1, nb_labels + 1))
    ordered = np.sort(sizes, axis=0)
    choicelist = []
    # Selects the 8 longest elements, binary closes to form longer elements and uses that as mask.
    # If there are fewer than 8 elements it uses 4 or 2 elements
    if len(ordered) >= 8:
        max_label = np.where(sizes >= ordered[-8])[0] + 1
        for a in max_label:
            choicelist.append(np.asarray(label_im == a))
            a1 = np.logical_or(choicelist[0], choicelist[1])
            a2 = np.logical_or(choicelist[2], choicelist[3])
            a3 = np.logical_or(choicelist[4], choicelist[5])
            a4 = np.logical_or(choicelist[6], choicelist[7])
            a5 = np.logical_or(a1, a2)
            a6 = np.logical_or(a3, a4)
            output = np.logical_or(a5, a6)
    elif 4 <= len(ordered) <= 7:
        max_label = np.where(sizes >= ordered[-4])[0] + 1
        for a in max_label:
            choicelist.append(np.asarray(label_im == a))
            a1 = np.logical_or(choicelist[0], choicelist[1])
            a2 = np.logical_or(choicelist[2], choicelist[3])
            output = np.logical_or(a1, a2)
    elif len(ordered) < 4:
        try:
            max_label = np.where(sizes >= ordered[-2])[0] + 1

```

```

        for a in max_label:
            choicelist.append(np.asarray(label_im == a))
        output = np.logical_or(choicelist[0], choicelist[1])
    except:
        output = elevation_map
    else:
        output = elevation_map
    elevation_map = nd.binary_closing(output, s, iterations=2)
    # Convert canny binary array to use as mask in segmentation
    elevation_map = elevation_map.astype(int)
    # Watershed function floods regions from markers
    segmentation = skimage.segmentation.watershed(
        evened,
        get_markers(
            image_array,
            evened_selem_size=markers_meanded_selem_size,
            markers_contrast_times=markers_contrast_times,
            markers_sd=markers_sd,
        ),
        mask=elevation_map,
        connectivity=1,
    )
    # Fill smaller holes within regions - usually due to noise; Label each region with different value
    segmentation = nd.binary_fill_holes(segmentation)
    label = sme.label(segmentation)
    if return_mask == True:
        return elevation_map
    else:
        return label

# Measures labelled regions of images which correspond to grains
# and uses selctive techniques to define whether each grain is an actual grain
def measure_props(
    path: Path,
    sigma: float = 3.8,
    canny_lt: float = 25,
    canny_ht: float = 5,
    dilated_contrast_times=8,
    evened_contrast_times=1,
    dilated_evened_selem_size: int = 9,
    evened_selem_size: int = 4,
    dilation_selem_size: int = 4,
    markers_meanded_selem_size: int = 4,
    markers_contrast_times: float = 30,
    markers_sd: float = 0.25,
    use_longest_elements: bool = False,
    iterations=1,
):

```

```

# Image opened, converted to greyscale and dimensions measured for
# calculating image:grain ratios, which allows for different size images
img = Image.open(path).convert("L")
w, h = img.size
area = w * h
tempdf = pd.DataFrame()
a = segment(
    img,
    sigma=sigma,
    canny_lt=canny_lt,
    canny_ht=canny_ht,
    dilated_contrast_times=dilated_contrast_times,
    evened_contrast_times=evened_contrast_times,
    dilated_evened_selem_size=dilated_evened_selem_size,
    evened_selem_size=evened_selem_size,
    dilation_selem_size=dilation_selem_size,
    markers_meaned_selem_size=markers_meaned_selem_size,
    markers_contrast_times=markers_contrast_times,
    markers_sd=markers_sd,
    use_longest_elements=use_longest_elements,
    iterations=iterations,
)
for region in sme.regionprops(a):
    # Filter out regions which take up less than 1% of the image or if the whole image has been de
    tected,
    # and uses a circularity factor to removes erroneous regions which have been detected
    if (
        1 > (region.area / area) > 0.001
        and (region.perimeter ** 2) / (4 * np.pi * region.area) < 3.5
    ):
        # Get region bounding box, apply a mask to grayscale image so only region
        # with grain in it has GLCM functions applied to it as all other values are 0
        uint_img: np.ndarray = imread(path, as_gray=True)
        uint_img = img_as_ubyte(uint_img)
        coord: Tuple[int, int, int, int] = region.bbox
        b = np.where(a, uint_img, 0)
        boxx = b[coord[0] : coord[2], coord[1] : coord[3]]
        # GLCM measures the differences of each pixel to its neighbouring pixels.
        # Sets inputs for GLCM relating to how each value is measured.
        # Distances = nth neighbour; angles = which directions (all)
        distances = np.array([1])
        angles = np.array(
            [0, np.pi / 8, np.pi / 4, 3 * (np.pi / 8), np.pi / 2, 5 * (np.pi / 8),
            3 * (np.pi / 4), 7 * (np.pi / 8)])
        # Creates GLCM and measures properties thereof, appends to DataFrame
        glcm = sft.greycomatrix(boxx, distances, angles, levels=256, normed=True)
        correlation = sft.greycoprops(glcm, prop="correlation")[0].mean()
        contrast = sft.greycoprops(glcm, prop="contrast")[0].mean()
        dissimilarity = sft.greycoprops(glcm, prop="dissimilarity")[0].mean()

```

```

homogeneity = sft.greycoprops(glcm, prop="homogeneity")[0].mean()
ASM = sft.greycoprops(glcm, prop="ASM")[0].mean()
energy = sft.greycoprops(glcm, prop="energy")[0].mean()
entropy = sme.shannon_entropy(boxx)
# prediction_model = pickle.load(open("errors_model3.sav", "rb"))
values = [h,region.area,region.perimeter,region.solidity,region.major_axis_length,
          region.minor_axis_length,region.equivalent_diameter,region.eccentricity,
          region.extent,region.filled_area,region.euler_number,entropy,contrast,
          dissimilarity,homogeneity,ASM,energy,correlation,region.bbox]
series = pd.Series(
    values,
    name=path.stem,
    index=[
        "img_h",
        "Area",
        "Perimeter",
        "Solidity",
        "Major_axis_length",
        "Minor_axis_length",
        "Equivalent_diameter",
        "Eccentricity",
        "Extent",
        "Filled_area",
        "Euler_number",
        "Entropy",
        "Contrast",
        "Dissimilarity",
        "Homogeneity",
        "ASM",
        "Energy",
        "Correlation",
        "Bounding_box",
    ],
)
# print(values)
# value = prediction_model.predict(
#     np.array(
#         [
#             region.area / (h / 100),
#             region.perimeter / (h / 100),
#             region.solidity,
#             region.major_axis_length / (h / 100),
#             region.minor_axis_length / (h / 100),
#             region.equivalent_diameter / (h / 100),
#             region.eccentricity,
#             region.extent,
#             entropy,
#             contrast,
#             dissimilarity,

```

```

        #             homogeneity,
        #             ASM,
        #             energy,
        #             correlation,
        #         ]
        #     ).reshape(1, -1)
        # )
        # print(value)
        # if value == 0:
        tempdf = tempdf.append(series)
        # else:
        #     continue
    else:
        continue
if tempdf.empty:
    return tempdf
# elif tempdf["Area"].size > 2:
#     tempdf = tempdf[(np.abs(stats.zscore(tempdf["Area"]))) < 1.5]
#     tempdf["thresh_factor"] = tempdf["Area"].min() / tempdf["Area"].max()
#     tempdf = tempdf.loc[tempdf["thresh_factor"] > 0.3].drop("thresh_factor", axis=1)
return tempdf

# Primary function for segmentation and measuring properties of grains.
# Runs measure_props, but if no grain returned, runs again but selects longest elements for
# mask rather than just canny filter. Works well for images with large grains.
def get_props(
    path: Path,
    sigma=1,
    canny_lt=75,
    canny_ht=5,
    dilated_contrast_times=8,
    evened_contrast_times=1,
    dilated_evened_selem_size=9,
    evened_selem_size=9,
    dilation_selem_size=4,
    markers_meaned_selem_size=3,
    markers_contrast_times=30,
    markers_sd=0.25,
    iterations=1,
):
    a = measure_props(path)
    if a.empty:
        return measure_props(
            path,
            sigma=sigma,
            canny_lt=canny_lt,
            canny_ht=canny_ht,
            dilated_contrast_times=dilated_contrast_times,

```

```

        evened_contrast_times=evened_contrast_times,
        dilated_evened_selem_size=dilated_evened_selem_size,
        evened_selem_size=evened_selem_size,
        dilation_selem_size=dilation_selem_size,
        markers_meaned_selem_size=markers_meaned_selem_size,
        markers_contrast_times=markers_contrast_times,
        markers_sd=markers_sd,
        use_longest_elements=True,
        iterations=iterations,
    )
else:
    return a

# Function to show transformations at each stage of measure_props for each image in dataset
def plot_analysis_on_images(inputfolder: Path):
    measuresdf = pd.DataFrame(
        columns=[
            "Area",
            "Perimeter",
            "Major_axis_length",
            "Minor_axis_length",
            "Equivalent_diameter",
            "Eccentricity",
            "Extent",
            "Contrast",
            "Dissimilarity",
            "Homogeneity",
            "ASM",
            "Energy",
            "Correlation",
            "Bounding_box",
        ]
    )
    for i, path in enumerate(inputfolder.glob("*.jpg")):
        uint_img = imread(path, as_gray=True)
        output = get_props(path)
        measuresdf = measuresdf.append(output)
        # Matplotlib configurations to show 5 images of different stages of the process
        inp = Image.open(path).convert("L")
        plt.fig, (ax1, ax2, ax3, ax4, ax5) = plt.subplots(
            nrows=1, ncols=5, figsize=(10, 15)
        )
        ax1.imshow(inp, cmap="gray")
        ax1.axis("off")
        ax1.set_title("Raw Image")
        ax2.imshow(
            inc_contrast(
                inp,

```

```

    ),
    cmap="gray",
)
ax2.axis("off")
ax2.set_title("Contrasted Image")
ax3.imshow(
    get_markers(
        inp,
    ),
    cmap="gray",
)
ax3.axis("off")
ax3.set_title("Markers")
ax4.imshow(
    segment(
        inp,
        return_mask=True,
        use_longest_elements=True,
    ),
    cmap="gray",
)
ax4.axis("off")
ax4.set_title("Mask")
ax5.imshow(
    segment(
        inp,
        use_longest_elements=True,
    ),
    cmap="gray",
)
ax5.axis("off")
ax5.set_title("Labelled")
plt.show(plt.fig)
# Bounding boxes are displayed seperately to observe the specific grains identified by the pro
gram
try:
    cols = 2
    fig = plt.figure(figsize=(20, 2), constrained_layout=True)
    gs = fig.add_gridspec(1, output["Bounding_box"].size)
    for i, _ in enumerate(output["Bounding_box"]):
        b = uint_img[
            output["Bounding_box"][i - 1][0] : output["Bounding_box"][i - 1][2],
            output["Bounding_box"][i - 1][1] : output["Bounding_box"][i - 1][3],
        ]
        cols += 1
        fig.add_subplot(gs[0, i:], title="BBox" + f"{i}").axis("off")
        plt.imshow(b)
    plt.show()
except:

```

```

        continue

    pd.set_option("display.max_columns", 500)
    # For use in IPython notebooks
    display(output)

# Optimises combination of input paramaters in terms to get the highest efficacy of positive grain det
# ection
def optimise_segmentation_variables(inputfolder, test_mask_thresholding=False):
    # DataFrame containing magnifications, size in micrometers, and actual number of grains in image
    actualcount = pd.read_csv(
        r"C:/Users/pinto/OneDrive - University of Edinburgh/coding/true_values.csv",
        names=["Image", "count", "size", "mag"],
        index_col="Image",
    )
    actualcountdf = pd.DataFrame(actualcount)
    # Each input variable is run through a range of values
    for path in inputfolder.glob("*.jpg"):
        size_efficiencydf = pd.DataFrame(columns=["accuracy", "SD_diff"])
        for sigma in np.linspace(1, 3, num=1):
            for canny_lt in np.linspace(100, 125, num=1):
                for canny_ht in np.linspace(5, 55, num=1):
                    for evened_contrast_times in np.linspace(10, 15, num=1):
                        for dilated_contrast_times in np.linspace(3, 6, num=1):
                            for dilated_evened_selem_size in np.linspace(1, 5, num=1):
                                for dilation_selem_size in np.linspace(1, 5, num=1):
                                    for iterations in np.linspace(1, 3, num=3):
                                        output = get_props(
                                            path,
                                            sigma=sigma,
                                            canny_lt=canny_lt,
                                            canny_ht=canny_ht,
                                            dilated_contrast_times=dilated_contrast_times,
                                            evened_contrast_times=evened_contrast_times,
                                            dilated_evened_selem_size=dilated_evened_selem_size,
                                            dilation_selem_size=dilation_selem_size,
                                            iterations=iterations,
                                        )
                                        if output.empty:
                                            sd_diff_40x = np.nan
                                            sd_diff_100x = np.nan
                                            size_series = pd.Series(
                                                [
                                                    sd_diff_40x,
                                                    sd_diff_100x,
                                                ],
                                                index=["accuracy", "SD_diff"],
                                                name=(

```

```

        f"{sigma},{canny_lt},{canny_ht},{evened_contrast_t
imes},{dilated_contrast_times},{dilated_evened_selem_size},{dilation_selem_size}"
    ),
)
size_efficacydf = size_efficacydf.append(
    size_series
)
# The size, magnification and actual cumber of grains in the i
mage are input manually.
# Double validation - must match the size and pass filters to
be considered a grain,
# then count calculated as percentage of actual grains in the
image
else:
    if (
        actualcountdf["mag"][f"{path.stem}"]
        == "40x"
    ):
        xlist = []
        for a in output["Major_axis_length"]:
            sd_diff_40x = (
                (
                    a
                    / int(
                        actualcountdf[
                            "size"
                        ][f"{path.stem}"]
                    )
                )
                - 0.686419
            ) / 0.273305
            if abs(sd_diff_40x) <= 2:
                xlist.append(sd_diff_40x)
    if (
        actualcountdf["mag"][f"{path.stem}"]
        == "100x"
    ):
        xlist = []
        for a in output["Major_axis_length"]:
            sd_diff_100x = (
                (
                    (a)
                    / int(
                        actualcountdf[
                            "size"

```

```

        ][f"{path.stem}"]
    )
    )
    - 2.703794
    )
    ) / 0.371814
    if abs(sd_diff_100x) <= 2:
        xlist.append(sd_diff_100x)
a = (len(xlist) * 100) / (
    int(
        actualcountdf["count"][
            f"{path.stem}"
        ]
    )
)
size_series = pd.Series(
    [a, xlist],
    index=["accuracy", "SD_diff"],
    name=(
        f"{sigma},{canny_lt},{canny_ht},{evened_contrast_t
imes},{dilated_contrast_times},{dilated_evened_selem_size},{dilation_selem_size}"
    ),
)
# Appended to full DataFrame containing % efficacy for eac
h grain,
# with each index containing the combination of input vari
ables
size_efficacydf = size_efficacydf.append(
    size_series
)
size_efficacydf.to_csv(f"size_{path.stem}.csv")

if __name__ == "__main__":
    # Global variables
    inputfolder = Path("Saps excuded/Included SAPS")
    plot_analysis_on_images(inputfolder)

```